

B. Tech - III Year – V Semester

Artificial Intelligence

Unit

4

Logic & Knowledge Representation: First order logic. Inference in first order logic, propositional vs. first order inference, Unification & lifts forward chaining, Backward chaining, Resolution, learning from observation, Explanation-based learning, Statistical Learning methods, and Reinforcement Learning.

Proposition Logic

Propositional Logic:

The smallest element is a **proposition**, which is a complete statement that is either **true or false**.

Example:

- “It is raining.” (Can only be true or false)
- It is less expressive – it can only represent simple facts.
- It **cannot represent** relationships between objects or handle multiple entities.

Example:

- “The light is on” (just a single fact)

Combines propositions using logical connectives like:

- AND ($\wedge$), OR ($\vee$), NOT ($\neg$), IMPLIES ($\rightarrow$)

• Example:

$P \rightarrow Q \rightarrow$ “If it rains, then the ground gets wet.”

First order

Logic

First Order Logic also known as Predicate logic, is a formal system used in **artificial intelligence, mathematics and logic** to represent relationships between objects and make inferences. It extends **Propositional Logic** by incorporating **quantifiers, predicates and variables**, making it more expressive.

- **Components of Predicate Logic**

- 1. **Constants (Objects)**: Represent specific entities in the domain. Example: *Alice, 5, Paris*

- 2. **Variables**: Represent unspecified elements in the domain. Example: *x, y, z*

- 3. **Predicates (Relations/Properties)**: Functions that define a property or a relation between objects. Example: *Loves(Alice, Bob)* (Alice loves Bob)

- 4. **Functions**: Define deterministic mappings from objects to objects. Example: *Father(John) = Robert* (John's father is Robert)

- 5. **Logical Connectives**: $\neg$ (Negation), $\wedge$ (AND), $\vee$ (OR), $\rightarrow$ (Implication), $\leftrightarrow$ (Biconditional)

- 6. **Quantifiers**:

- 1. **Universal Quantifier ($\forall$)**: "For all" – expresses generality. Example: $\forall x \text{ Loves}(x, \text{Chocolate}) \rightarrow$ (Everyone loves chocolate)

- 2. **Existential Quantifier ($\exists$)**: "There exists" – expresses existence. Example: $\exists x \text{ Loves}(x, \text{Pizza})$ (At least one person loves pizza)

First Order

Logic

- Examples of Predicate Logic Sentences

1. Basic Statements

- "John is a student." $\rightarrow$ Student(John)
- "Mary is a doctor." $\rightarrow$ Doctor(Mary)

2. Relationships Between Objects

- "Alice loves Bob." $\rightarrow$ Loves(Alice, Bob)
- "Tom is taller than Jerry." $\rightarrow$ Taller(Tom, Jerry)
- "Paris is the capital of France." $\rightarrow$ Capital(Paris, France)

• 3. Universal Quantifier ($\forall$) – "For all"

- "All humans are mortal." $\rightarrow \forall x (\text{Human}(x) \rightarrow \text{Mortal}(x))$
- "Every student studies." $\rightarrow \forall x (\text{Student}(x) \rightarrow \text{Studies}(x))$
- "If a person is a teacher, they teach." $\rightarrow \forall x (\text{Teacher}(x) \rightarrow \text{Teaches}(x))$

• 4. Existential Quantifier ($\exists$) – "There exists"

- "Some people like ice cream." $\rightarrow \exists x (\text{Person}(x) \wedge \text{Likes}(x, \text{IceCream}))$
- "There exists a person who is a millionaire." $\rightarrow \exists x (\text{Person}(x) \wedge \text{Millionaire}(x))$
- "Some dogs are friendly." $\rightarrow \exists x (\text{Dog}(x) \wedge \text{Friendly}(x))$

Inference in first order

Inference in First-Order Logic (FOL) involves deriving new sentences or conclusions from a set of given premises or statements using logical reasoning. It relies on the formal structure of FOL, including variables, constants, predicates, functions, and quantifiers, to construct valid arguments. Key inference rules, such as **Universal Generalization**, **Existential Introduction**, **Existential Instantiation** and **Universal Instantiation**, are applied to manipulate these sentences.

Automated theorem proving systems leverage these principles to systematically apply inference rules and derive logical conclusions, making inference in FOL a fundamental aspect of logical reasoning and AI. This process of AI inference ensures robust decision-making and logical consistency within artificial intelligence systems.

Substitution

Substitution is a fundamental operation in first-order logic that involves replacing a variable with a specific constant or phrase within an expression. This process is essential for effectively applying inference rules.

For instance, writing $F[a/x]$ indicates that the constant "a" replaces the variable "x".

Equality: In First-Order Logic, we can form sentences using predicates, terms, and also equality symbols (=).

Equality shows that two terms refer to the same object in the domain.

Example:

Father(John) = Smith

- This means that the object referred to by Father(John) is the same as the object called Smith.

In other words, Smith is John's father.

Using Negation:

- We can also use negation ($\neg$) to show that two terms are not equal.

- Example:

- $\neg(\text{Father}(\text{John}) = \text{Smith}) \rightarrow$ means *Smith is not John's father*.

- It is useful in logical reasoning, mathematical proofs, and AI knowledge representation.

First-Order Logic Inference Rules for Quantifiers:

First-order logic (FOL) includes a set of inference rules that govern how we can manipulate and infer new information from statements involving quantifiers.

Here are the primary inference rules for quantifiers in FOL, explained in detail:

1. Universal Generalization
2. Universal Instantiation
3. Existential Instantiation
4. Existential introduction

Universal Generalization

Universal Generalization is a rule of inference in First-Order Logic (FOL).

- It allows us to move from a **specific case** to a **general statement** about all cases.
- In simple terms, if a property is true for an **arbitrary example**, then it is true for **everything** in that domain.

Formal Representation

If a statement $P(c)$ is true for an **arbitrary element** c ,
then we can conclude that $P(x)$ is true for **all** x .

Symbolic Representation

$$\frac{P(c)}{\forall x P(x)}$$

- The chosen element c must be **arbitrary**, not special or unique.

Example

Goal: Prove that *every number greater than 0 is positive*.

Step 1 – Premise:

Let c be an arbitrary number.

If $c > 0$, then c is **positive**.

$c > 0 \rightarrow c$ is positive

Step 2 – Apply UG:

Since c was arbitrary,

$\forall x(x > 0 \rightarrow x \text{ is positive})$

Conclusion:

Therefore, *for all x , if $x > 0$, then x is positive*.

Universal Instantiation

Universal Instantiation (also called as universal elimination or UI) is a valid inference rule. It can be applied multiple times to add new sentences. The new knowledge base (KB) is logically equivalent to the previous KB.

As per **UI**, we can infer any sentence obtained by substituting a ground term for the variable. The **UI** rule states that we can infer any sentence $P(c)$ by substituting a ground term c (a constant within domain x) from $\forall x P(x)$ for any object in the universe of discourse.

Example:

Suppose we know that all humans are mortal, i.e., $\forall x (\text{Human}(x) \rightarrow \text{Mortal}(x))$. By using universal instantiation, we can infer that if Socrates is human, then Socrates is mortal.

1. Premise: $\forall x (\text{Human}(x) \rightarrow \text{Mortal}(x))$

2. Conclusion: $\text{Human}(\text{Socrates}) \rightarrow \text{Mortal}(\text{Socrates})$

Existential Instantiation or elimination

Existential Instantiation is a **valid inference rule** in **first-order logic**.

It allows us to **replace an existentially quantified variable** ($\exists x$) with a **new constant symbol** (c).

$\exists x P(x)$ means “There exists some x such that $P(x)$ is true.” In simple words At least one thing has the property P .

The **Existential Instantiation rule** helps us take this general statement and talk about **a specific example** of it means from $\exists x P(x)$, we can write $P(c)$, where c is a new name (a constant) for that “some x ”.

The constant c must be **new**, meaning it doesn’t refer to anything else in your knowledge base.

We don’t know *who* or *what* x is — we just know *someone* exists. So we give that someone a temporary name, like c , **Ram**, or a_1 , just to reason about them.

Existential Instantiation

After replacing $\exists x$ with c ,
the **new sentence** ($P(c)$) is not exactly the same as the old one ($\exists x P(x)$),
but it is still **consistent** — it won't make the knowledge base false if the old one was true

Example

Original statement:

$\exists x (\text{Student}(x) \wedge \text{Studies}(x, \text{AI})) \rightarrow$ Means “There exists at least one student who studies AI.”

Apply **Existential Instantiation**:

$(\text{Student}(\text{sita}) \wedge \text{Studies}(\text{sita}, \text{AI}))$

We replaced x with sita (a new name). Now we can talk about “ sita ” as that specific student — even though we don't really know who it is.

Existential introduction

It is also called **Existential Generalization**. This is a **valid inference rule** in **first-order logic** that goes in the **opposite direction** of Existential Instantiation. That means, If a particular thing (say, c) has a property P , then we can conclude that **something** ($\exists x$) has that property.

From

$$P(c)$$

we can infer

$$\exists x P(x)$$

Example:

Priyansu got good marks in English.

→ This means: $P(\text{Priyansu})$ is true.

So we can infer:

“Someone got good marks in English.” → $\exists x P(x)$

Here, we moved from a **specific case (Priyansu)** to a **general existential statement (someone)**.

Inference in first order

10 give Universal Generalization

- Universal generalization is a valid inference rule which states that if premise $P(c)$ is true for any arbitrary element c in the universe of discourse, then we can have a conclusion as $\forall x P(x)$.

- It can be represented as:
$$\frac{P(c)}{\forall x P(x)}$$

• Universal Instantiation

- Universal instantiation is also called as universal elimination or UI is a valid inference rule. It can be applied multiple times to add new sentences.

- It can be represented as:
$$\frac{\forall x P(x)}{P(c)}$$

• Existential Instantiation

- Existential instantiation is also called as Existential Elimination, which is a valid inference rule in first-order logic.

- It can be represented as:
$$\frac{\exists x P(x)}{P(c)}$$

• Existential introduction

- An existential introduction is also known as an existential generalization, which is a valid inference rule in first-order logic.

- It can be represented as:
$$\frac{P(c)}{\exists x P(x)}$$

Propositional vs. First Order Inference

Feature	Propositional Logic	First-Order Logic
Basic Unit	Propositions	Predicates, constants, variables
Expressiveness	Limited to true/false statements	Expressive, can represent relationships and properties. Rich, detailed reasoning about objects and their relationships
Quantifiers	None	Universal ($\forall$) and Existential ($\exists$)
Syntax	Combine propositions with logical connectives like AND,OR,NOT	Uses predicates and quantifiers
Semantics	Evaluated using Truth tables	Interpretation over a domain of objects

Unificatio

Unification in Artificial Intelligence (AI) is a key concept in first-order logic and automated reasoning. It is mainly used in inference algorithms like resolution, forward chaining, and backward chaining to make logical deductions.

- It is the process of making different expressions or terms identical by assigning values to variables in a way that allows them to match or unify.

Unification is a key component of all first order inference algorithms.

Unification helps automated reasoning systems (like Prolog or theorem provers) to:

- Match rules with facts
- Perform logical inference
- Enable backward and forward chaining

Unification Process in First-Order Logic

Given two atomic formulas:

1. $P(x,y)$

2. $P(a,z)$

We aim to unify them by finding a substitution θ . Here, the unifier $\theta = \{x/a, y/z\}$ transforms both formulas into:

$P(a,z)$

This unification allows reasoning systems to proceed with proofs or deductions.

Unificatio

Following are some basic conditions for unification:

- Predicate symbol must be same, atoms or expression with different predicate symbol can never be unified.
- No. of arguments in both expressions must be identical
- Unification will fail if there are two similar variables present in the same expression.
- Example
 - **UNIFY(Knows(John,x), Knows(John, kumar))**
SUBST $\square = x/kumar$
 $\psi(Knows(John,kumar), Knows(John, kumar))$ Successfully unified
 - **UNIFY(Knows(John,x), Knows(x, kumar))**
Unification failed because of same variable in both expression

Unificatio

Example

- **UNIFY(Knows(John,x), Knows(y, kumar))**

Compare predicates Both have the same predicate name Knows .

So, we now unify their arguments:

Step 2: Substitute values

- From the first pair:

Substitute y with John $\rightarrow$ (SUBST₁ = {y/John})

From the second pair:

Substitute x with kumar $\rightarrow$ (SUBST₂ = {x/kumar})

Step 3: Combine substitutions

Now apply both substitutions together:

Final Unification Substitution:

$\theta = \{x/kumar, y/John\}$

When we apply θ to both sides:

Knows(John, x) $\rightarrow$ Knows(John, kumar)

Knows(y, kumar) $\rightarrow$ Knows(John, kumar)

Both are identical.

Applications of Unification

Natural Language Processing

(NLP): Unification is used in NLP for various tasks, such as parsing and semantic analysis.

- In parsing, unification helps identify the relationships between words in a sentence, allowing the system to build syntactic and semantic structures.
- Unification is also essential for handling ambiguous language and constructs pronoun references resolving
- For example, unification can help determine that "he" refers to a specific person or entity mentioned earlier in a text.

- **Logic Programming:** Unification is a cornerstone of logic programming languages like Prolog. In logic programming, unification is used to match query predicates with database predicates.
- **Symbolic Reasoning:** In symbolic reasoning and theorem proving, unification is employed to determine whether two logical expressions are equivalent or if one can be transformed into the other by substituting values for variables.
- **Expert Systems:** Unification is used in expert systems to match user queries with the knowledge stored in the system's database. It helps determine which rules or pieces of information are relevant to a specific problem or query, facilitating the expert system's decision-making process.

Example of unification

Find the **MGU** (Most General Unifier) of $Q(f(x), y)$ and $Q(f(a), b)$

Step 1:

Both predicates are Q , so we compare arguments

First arguments: $f(x)$ and $f(a)$

Second arguments: y and b

Step 2:

To make $f(x)$ and $f(a)$ equal $\rightarrow$ we must have $x=a$

And to make y and b equal $\rightarrow$ we must have $y=b$

So, $\theta = \{x/a, y/b\}$

Step 3: Apply the substitution θ : $Q(f(x), y)\theta = Q(f(a), b)$

$Q(f(a), b)\theta = Q(f(a), b)$

Both are now identical, so unification succeeds.

Example 2

UNIFY(Knows(John,x), Knows(y,Mother(y)))

Step 1:

Predicates both (Knows) are same

Step 2:

Substitute y with **John**

Substitute x with **Mother(John)**

Step 3:

$\theta = \text{knows}(\text{John}, \text{Mother}(\text{John})), \text{Knows}(\text{John}, (\text{John}))$

Inference engine

- The inference engine is the component of the intelligent system in artificial intelligence, which applies logical rules to the knowledge base to infer new information from known facts.
- The first inference engine was part of the expert system.
- Inference engine commonly proceeds in two modes, which are:
 1. Forward chaining
 2. Backward chaining

Forward chaining

- Forward chaining is also known as a **forward deduction or forward reasoning** method when using an inference engine.
- Forward chaining is a form of reasoning which **start with atomic sentences in the knowledge base** and **applies inference rules** in the forward direction to **extract more data until a goal is reached**.
- The Forward-chaining algorithm starts from **known facts**, **triggers all rules whose premises are satisfied**, and **add their conclusion to the known facts**. This process repeats until the problem is solved.
- **Properties of Forward-Chaining:**
 - It is a bottom-up approach.
 - It is a process of making a conclusion based on known facts or data, by starting from the initial state and reaches the goal state.
 - Forward-chaining approach is also called as data-driven as we reach to the goal using available data.
 - Forward -chaining approach is commonly used in the expert system, such as CLIPS, business, and production rule systems.

Forward chaining

- Example:
 - Facts: It is raining
 - Rule: If it is raining, the street is wet
 - Conclusion: The Street is wet
- Conclude from A and $A \rightarrow B$ to B



Forward chaining

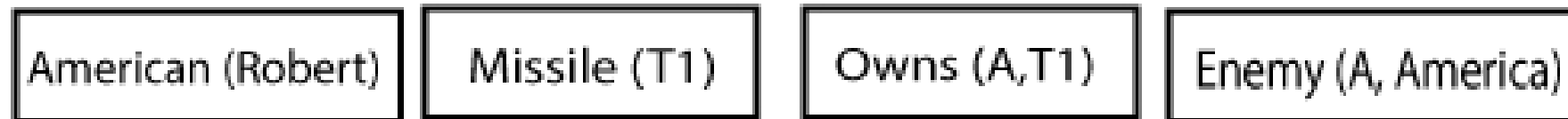
- Example:
- **"As per the law, it is a crime for an American to sell weapons to hostile nations. Country A, an enemy of America, has some missiles, and all the missiles were sold to it by Robert, who is an American citizen."**
- **Prove that "Robert is criminal."**

Forward chaining

- Facts Conversion into FOL:
- It is a crime for an American to sell weapons to hostile nations. (Let's say p, q, and r are variables)
 $\text{American}(p) \wedge \text{weapon}(q) \wedge \text{sells}(p, q, r) \wedge \text{hostile}(r) \rightarrow \text{Criminal}(p) \quad \dots(1)$
- Country A has some missiles. $\exists p \text{ Owns}(A, p) \wedge \text{Missile}(p)$. It can be written in two definite clauses by using Existential Instantiation, introducing new Constant T1.
 $\text{Owns}(A, T1) \quad \dots\dots(2)$
 $\text{Missile}(T1) \quad)$
- All of the missiles were sold to country A by Robert.
 $\forall p \text{ Missiles}(p) \wedge \text{Owns}(A, p) \rightarrow \text{Sells}(\text{Robert}, p, A) \quad \dots\dots(3)$
 $\quad) \quad \dots\dots(4)$
- Missiles are weapons.
 $\text{Missile}(p) \rightarrow \text{Weapons}(p) \quad \dots\dots(5)$
- Enemy of America is known as hostile.
 $\text{Enemy}(p, \text{America}) \rightarrow \text{Hostile}(p) \quad \dots\dots(6)$
 $\quad)$
 - Country A is an enemy of America.
 $\text{Enemy}(A, \text{America}) \quad \dots\dots(7)$
 $\quad)$
- Robert is American
 $\text{American}(\text{Robert}). \quad \dots\dots(8)$
 $\quad)$

Forward chaining

- Forward chaining proof:
- **Step-1:**
- we will start with the known facts and will choose the sentences which do not have implications, such as: **American(Robert)**, **Enemy(A, America)**, **Owns(A, T1)**, and **Missile(T1)**. All these facts will be represented as below.

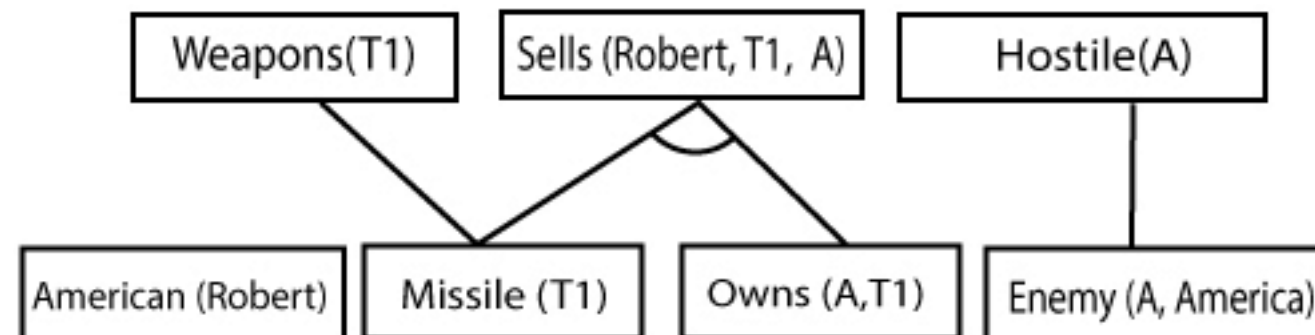


Step 2: we will see those facts which infer from available facts and with satisfied premises.

Rule-(1) does not satisfy premises, so it will not be added in the first iteration. Rule-(2) and (3) are already added.

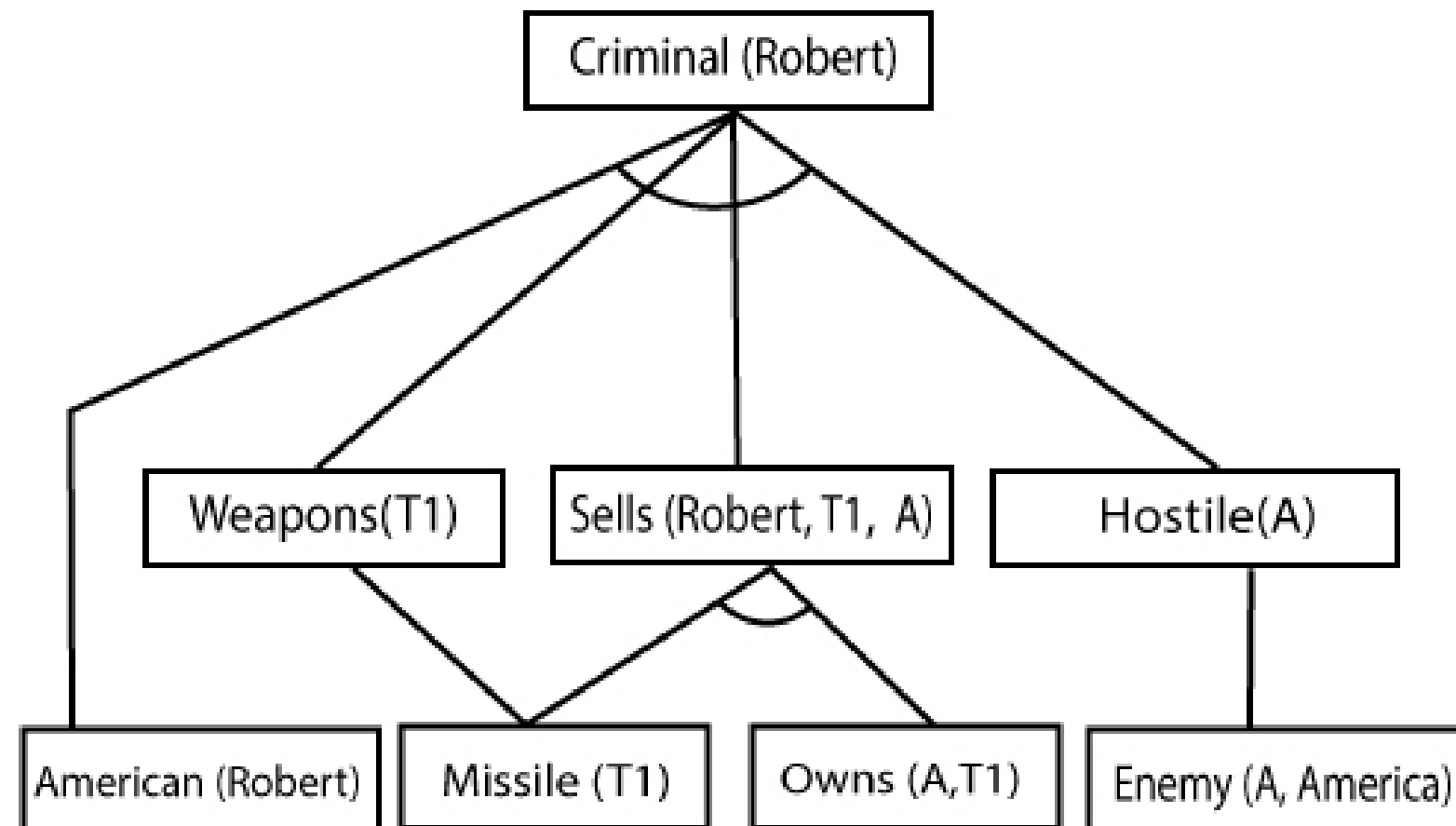
Rule-(4) satisfy with the substitution $\{p/T1\}$, so **Sells (Robert, T1, A)** is added, which infers from the conjunction of Rule (2) and (3).

Rule-(6) is satisfied with the substitution $\{p/A\}$, so **Hostile(A)** is added and which infers from Rule-(7).



Forward chaining

- **Step-3:**
- we can check Rule-(1) is satisfied with the substitution $\{p/\text{Robert}, q/T1, r/A\}$, so we can add **Criminal(Robert)** which infers all the available facts. And hence we reached our goal statement.



Hence it is proved that Robert is Criminal using forward chaining approach.

Backward chaining

- Backward-chaining is also known as a backward deduction or backward reasoning method when using an inference engine.
- **Properties of Backward-Chaining:**
 - It is known as a top-down approach.
 - Backward-chaining is based on modus ponens inference rule.
 - In backward chaining, the goal is broken into sub-goal or sub-goals to prove the facts true.
 - It is called a goal-driven approach, as a list of goals decides which rules are selected and used.
 - Backward -chaining algorithm is used in game theory, automated theorem proving tools, inference engines, proof assistants, and various AI applications.
 - The backward-chaining method mostly used a depth-first search strategy for proof.

Backward chaining

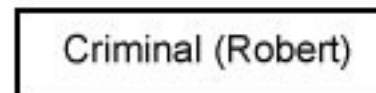
- Example:
- **"As per the law, it is a crime for an American to sell weapons to hostile nations. Country A, an enemy of America, has some missiles, and all the missiles were sold to it by Robert, who is an American citizen."**
- **Prove that "Robert is criminal."**

Backward chaining

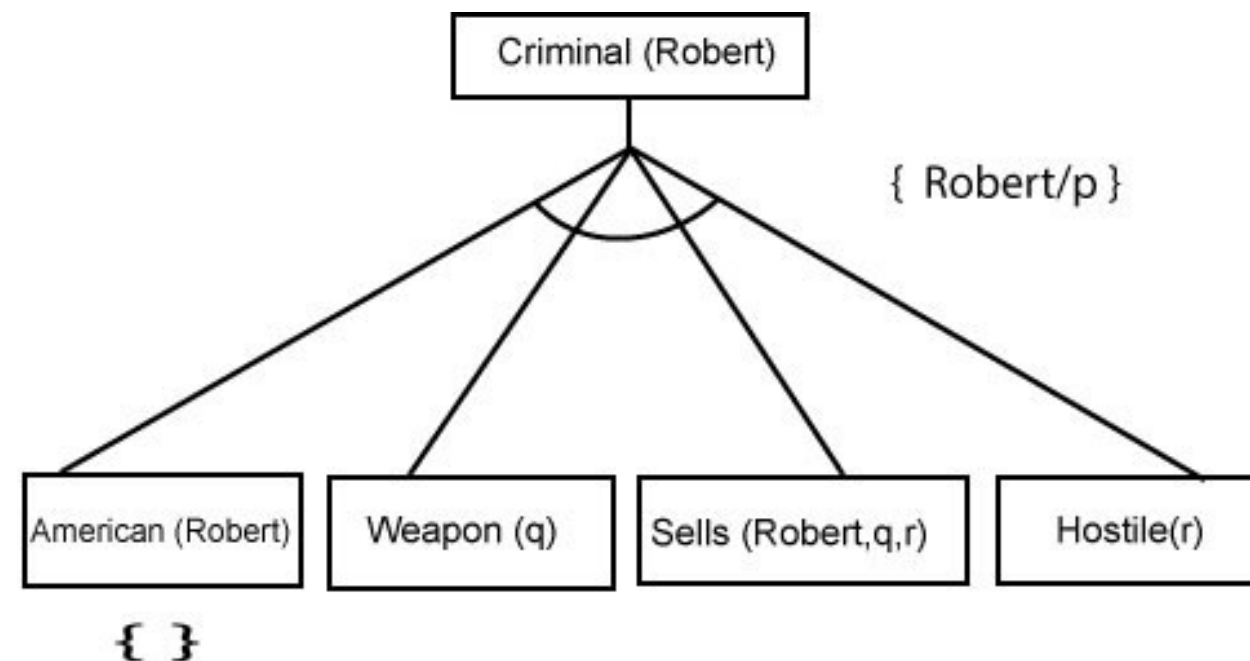
- Facts Conversion into FOL:
- It is a crime for an American to sell weapons to hostile nations. (Let's say p, q, and r are variables)
 $\text{American}(p) \wedge \text{weapon}(q) \wedge \text{sells}(p, q, r) \wedge \text{hostile}(r) \rightarrow \text{Criminal}(p) \quad \dots(1)$
- Country A has some missiles. $\exists p \text{ Owns}(A, p) \wedge \text{Missile}(p)$. It can be written in two definite clauses by using Existential Instantiation, introducing new Constant T1.
 $\text{Owns}(A, T1) \quad \dots\dots(2)$
 $\text{Missile}(T1) \quad)$
- All of the missiles were sold to country A by Robert.
 $\forall p \text{ Missiles}(p) \wedge \text{Owns}(A, p) \rightarrow \text{Sells}(\text{Robert}, p, A) \quad \dots\dots(3)$
 $\dots\dots(4)$
 $)$
- Missiles are weapons.
 $\text{Missile}(p) \rightarrow \text{Weapons}(p) \quad \dots\dots(5)$
- Enemy of America is known as hostile.
 $\text{Enemy}(p, \text{America}) \rightarrow \text{Hostile}(p) \quad \dots\dots(6)$
 $)$
 - Country A is an enemy of America.
 $\text{Enemy}(A, \text{America}) \quad \dots\dots(7)$
 $)$
- Robert is American
 $\text{American}(\text{Robert}). \quad \dots\dots(8)$
 $)$

Backward chaining

- Backward chaining proof:
- **Step-1:**
- we will take the goal fact. And from the goal fact, we will infer other facts, and at last, we will prove those facts true. So our goal fact is "Robert is Criminal," so following is the predicate of it.

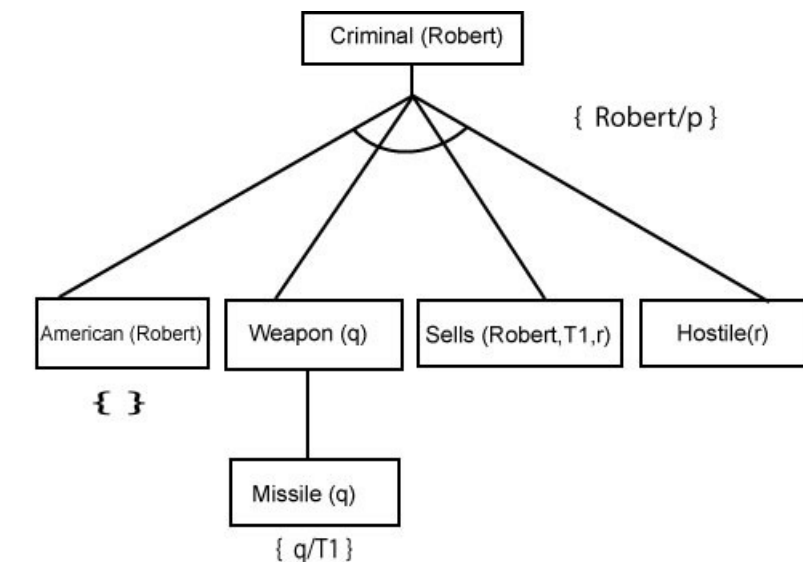


Step 2: we will infer other facts from goal fact which satisfies the rules. So as we can see in Rule-1, the goal predicate Criminal (Robert) is present with substitution $\{\text{Robert}/p\}$. So we will add all the conjunctive facts below the first level and will replace p with Robert.

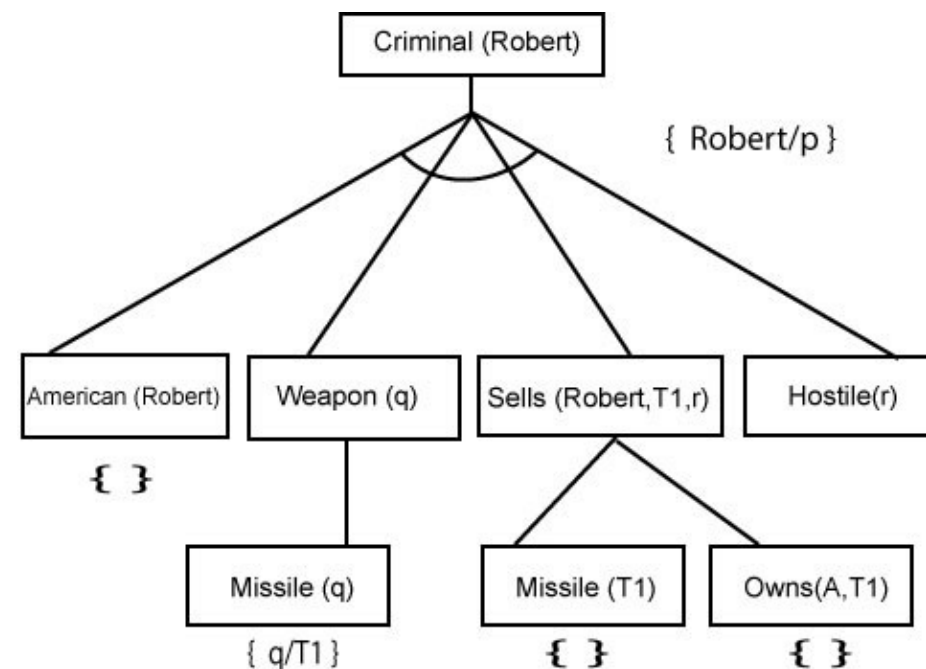


Backward chaining

- Backward chaining proof:
- **Step-3:** we will extract further fact Missile(q) which infer from Weapon(q), as it satisfies Rule-(5). Weapon (q) is also true with the substitution of a constant T1 at q.

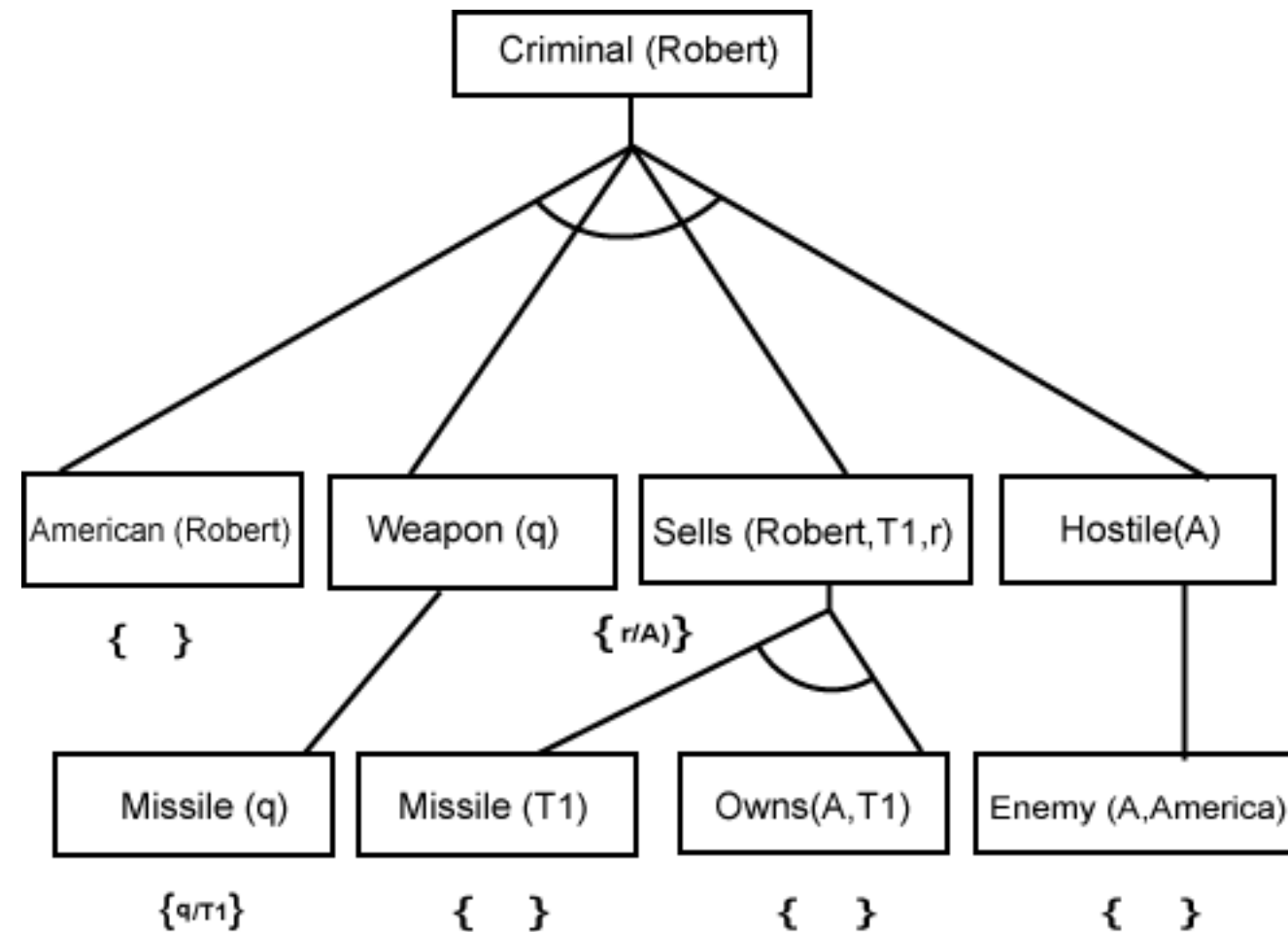


- Step-4, we can infer facts Missile(T1) and Owns(A, T1) from Sells(Robert, T1, r) which satisfies the **Rule- 4**, with the substitution of A in place of r. So, these two statements are proved here.



Backward chaining

- Backward chaining proof:
- At step-5, we can infer the fact **Enemy(A, America)** from **Hostile(A)** which satisfies Rule- 6. And hence all the statements are proved true using backward chaining.



Forward and Backward chaining

- Problems:
- Problem 1: As per medical law, it is a violation for a licensed doctor to prescribe banned drugs to patients. Drug X is a banned drug. Dr. Smith is a licensed doctor and has prescribed Drug X to a patient. (Using Forward Chaining)
- Problem 2: According to medical ethics, it is considered negligence if a doctor fails to treat an emergency patient without a valid reason. Patient B was in a critical condition and was denied treatment by Dr. Johnson. Dr. Johnson did not provide any valid reason for refusing treatment. (Using Forward and Backward Chaining)
- Problem 3: If a student has passed high school and scored above 90% in their final exams, they are eligible for admission. If a student is eligible for admission and has extracurricular achievements, they are shortlisted for an interview. If a student passes the interview, they are admitted to the university. (Using Forward and Backward Chaining)

Resolution

- Resolution is a theorem proving technique that proceeds by building refutation proofs, i.e., proofs by contradictions. It was invented by a Mathematician John Alan Robinson in the year 1965.
- Resolution is used, if there are various statements are given, and we need to prove a conclusion of those statements. Unification is a key concept in proofs by resolutions. Resolution is a single inference rule which can efficiently operate on the conjunctive normal form or clausal form.
- Clause: Disjunction of literals (an atomic sentence) is called a clause. It is also known as a unit clause.
- Conjunctive Normal Form: A sentence represented as a conjunction of clauses is said to be conjunctive normal form or CNF.

Resolution - Definition

- The resolution inference rule takes two premises in the form of clauses $(A \vee x)$ and $(B \vee \neg x)$ and gives the clause $(A \vee B)$ as a conclusion.
- Resolution proof technique works this way -
- (i) select two clauses that contain conflicting terms
- (ii) combine those two clauses and
- (iii) cancel out the conflicting terms

Example:

We can resolve two clauses which are given below:

$[\text{Animal}(g(x) \vee \text{Loves}(f(x), x)]$ and $[\neg \text{Loves}(a, b) \vee \neg \text{Kills}(a, b)]$

- Where two complimentary literals are:

$\text{Loves}(f(x), x)$ and $\neg \text{Loves}(a, b)$

- These literals can be unified with unifier $\theta = [a/f(x), \text{and } b/x]$, and it will generate a resolvent clause: $[\text{Animal}(g(x) \vee \neg \text{Kills}(f(x), x)]$.

The Resolution Steps

- Conversion of facts into first-order logic.
- Convert FOL statements into CNF
- Negate the statement which needs to prove (proof by contradiction)
- Draw resolution graph (unification)

Exempl

- John likes all kind of food.
- Apple and vegetable are food
- Anything anyone eats and not killed is food.
- Anil eats peanuts and still alive
- Harry eats everything that Anil eats.
- Prove by resolution that:
- John likes peanuts.

Step-1: Conversion of Facts into FOL – In the first step we will convert all the given statements into its first order logic

- a. $\forall x: \text{food}(x) \rightarrow \text{likes}(\text{John}, x)$
 - b. $\text{food}(\text{Apple}) \wedge \text{food}(\text{vegetables})$
 - c. $\forall x \forall y: \text{eats}(x, y) \wedge \neg \text{killed}(x) \rightarrow \text{food}(y)$
 - d. $\text{eats}(\text{Anil}, \text{Peanuts}) \wedge \text{alive}(\text{Anil})$.
 - e. $\forall x: \text{eats}(\text{Anil}, x) \rightarrow \text{eats}(\text{Harry}, x)$
 - f. $\forall x: \neg \text{killed}(x) \rightarrow \text{alive}(x)$
 - g. $\forall x: \text{alive}(x) \rightarrow \neg \text{killed}(x)$
 - h. $\text{likes}(\text{John}, \text{Peanuts})$
- } **added predicates.**

Exempl

- Step-2: Conversion of FOL into CNF
- In First order logic resolution, it is required to convert the FOL into CNF as CNF form makes easier for resolution proofs.

- Eliminate all implication ($\rightarrow$) and rewrite

$$\forall x \neg \text{food}(x) \vee \text{likes}(\text{John}, x) \vee \text{food}(\text{Apple}) \wedge \neg \text{food}(\text{vegetables}),$$

$$\forall x \forall y \neg [\text{eats}(x, y) \wedge \neg \text{killed}(x)] \vee \text{food}(y) \quad \forall x \forall y [(\text{eats}(x, y) \wedge \neg \text{killed}(x)) \rightarrow \text{food}(y)]$$

$$\text{eats}(\text{Anil}, \text{Peanuts}) \wedge \text{alive}(\text{Anil})$$

$$\forall x \neg \text{eats}(\text{Anil}, x) \vee \text{eats}(\text{Harry}, x) \quad \forall x [\text{eats}(\text{Anil}, x) \rightarrow \text{eats}(\text{Harry}, x)] \quad A \rightarrow B \rightarrow \sim A \vee B$$

$$\forall x \neg [\neg \text{killed}(x)] \vee \text{alive}(x) \quad \forall x [\neg \text{killed}(x) \rightarrow \text{alive}(x)]$$

$$\forall x \neg \text{alive}(x) \vee \neg \text{killed}(x) \text{ likes}(\text{John}, \text{Peanuts}).$$

Exempl

Move negation ($\neg$) inwards and rewrite

$\forall x \neg \text{food}(x) \vee \text{likes}(\text{John}, x)$
 $\text{food}(\text{Apple}) \wedge \text{food}(\text{vegetables})$
 $\forall x \forall y \neg \text{eats}(x, y) \vee \text{killed}(x) \vee \text{food}(y)$
 $\text{eats}(\text{Anil}, \text{Peanuts}) \wedge \text{alive}(\text{Anil})$
 $\forall x \neg \text{eats}(\text{Anil}, x) \vee \text{eats}(\text{Harry}, x)$
 $\forall x \neg \text{killed}(x) \vee \text{alive}(x)$
 $\forall x \neg \text{alive}(x) \vee \neg \text{killed}(x)$
 $\text{likes}(\text{John}, \text{Peanuts})$

Rename variables or standardize variables

$\forall x \neg \text{food}(x) \vee \text{likes}(\text{John}, x) \text{food}(\text{Apple})$
 $\wedge \text{food}(\text{vegetables})$
 $\forall y \forall z \neg \text{eats}(y, z) \vee \text{killed}(y) \vee \text{food}(z)$
 $\text{eats}(\text{Anil}, \text{Peanuts}) \wedge \text{alive}(\text{Anil})$
 $\forall w \neg \text{eats}(\text{Anil}, w) \vee \text{eats}(\text{Harry}, w)$
 $\forall g \neg \text{killed}(g) \vee \text{alive}(g)$
 $\forall k \neg \text{alive}(k) \vee \neg \text{killed}(k) \text{likes}(\text{John},$
 $\text{Peanuts}).$

Exempl

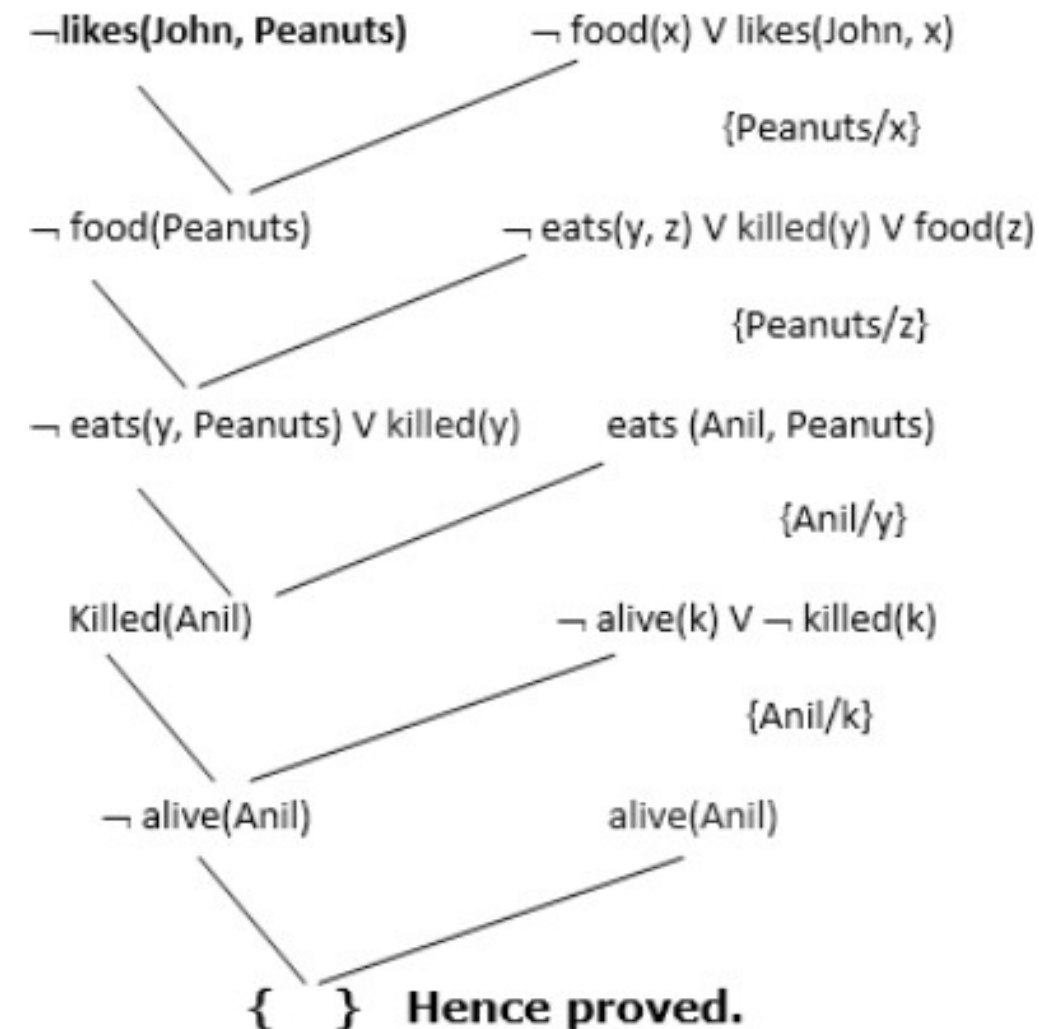
- Drop Universal quantifiers.
- In this step we will drop all universal quantifier since all the statements are not implicitly quantified so we don't need it.
 - $\text{food}(x) \vee \text{likes}(\text{John}, x) \text{ food}(\text{Apple})$
 $\text{food}(\text{vegetables})$
 - $\text{eats}(y, z) \vee \text{killed}(y) \vee \text{food}(z) \text{ eats } (\text{Anil}, \text{Peanuts})$
 $\text{alive}(\text{Anil})$
 - $\text{eats}(\text{Anil}, w) \vee \text{eats}(\text{Harry}, w) \text{ killed}(g) \vee \text{alive}(g)$
 - $\text{alive}(k) \vee \neg \text{killed}(k) \text{ likes}(\text{John}, \text{Peanuts}).$

Exempl

Step-3: Negate the statement to be proved – In this statement, we will apply negation to the conclusion statements, which will be written as
 $\neg \text{likes}(\text{John}, \text{Peanuts})$

Step-4: Draw Resolution graph:

- Now in this step, we will solve the problem by resolution tree using substitution. For the above problem, it will be given as follows:

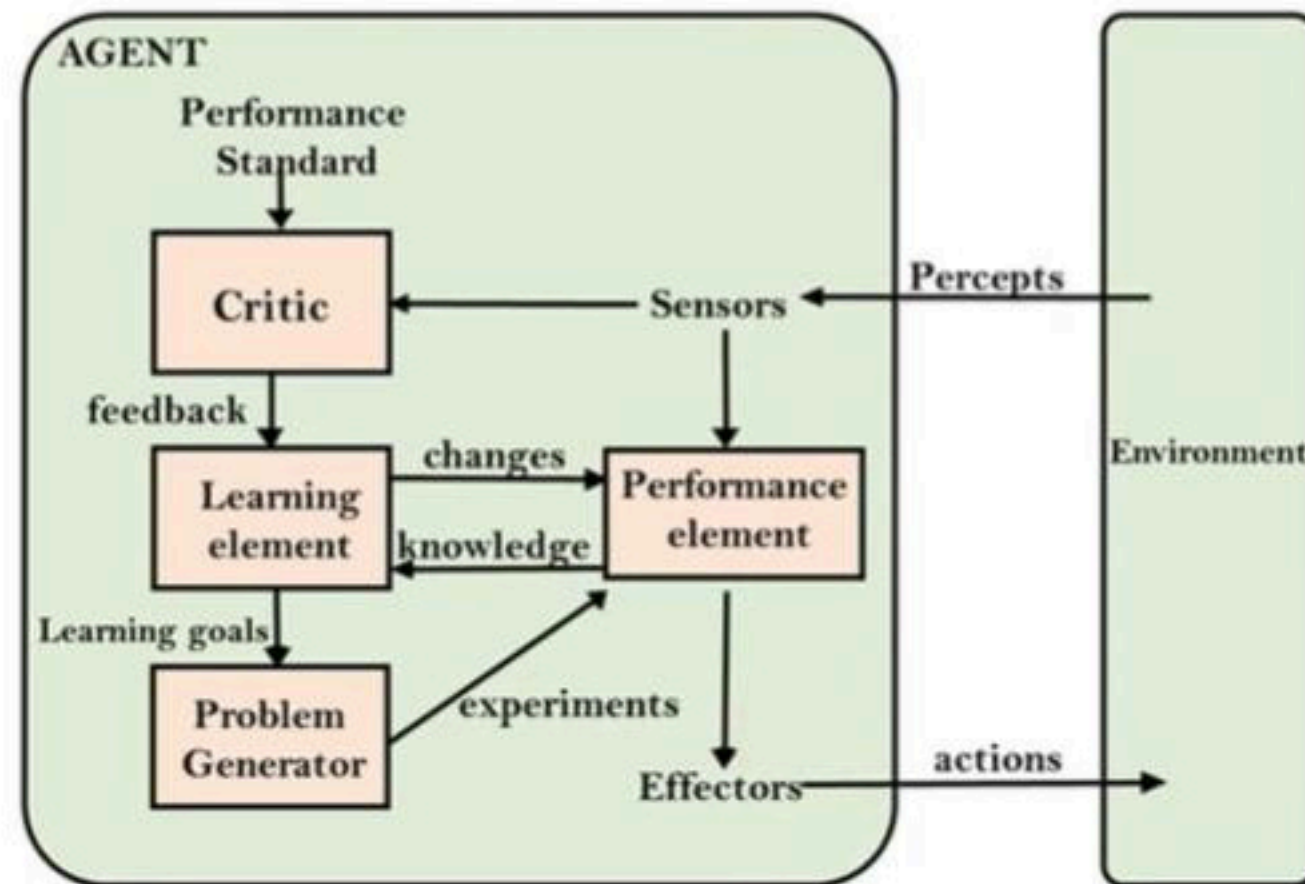


Learning

- **Learning** is Agent's percepts should be used for **acting**,
- It also used for improving the **agent's ability to act in the future.**
- Learning takes place as the agent **observes**, its interactions with the **world and its own decision-making processes.**

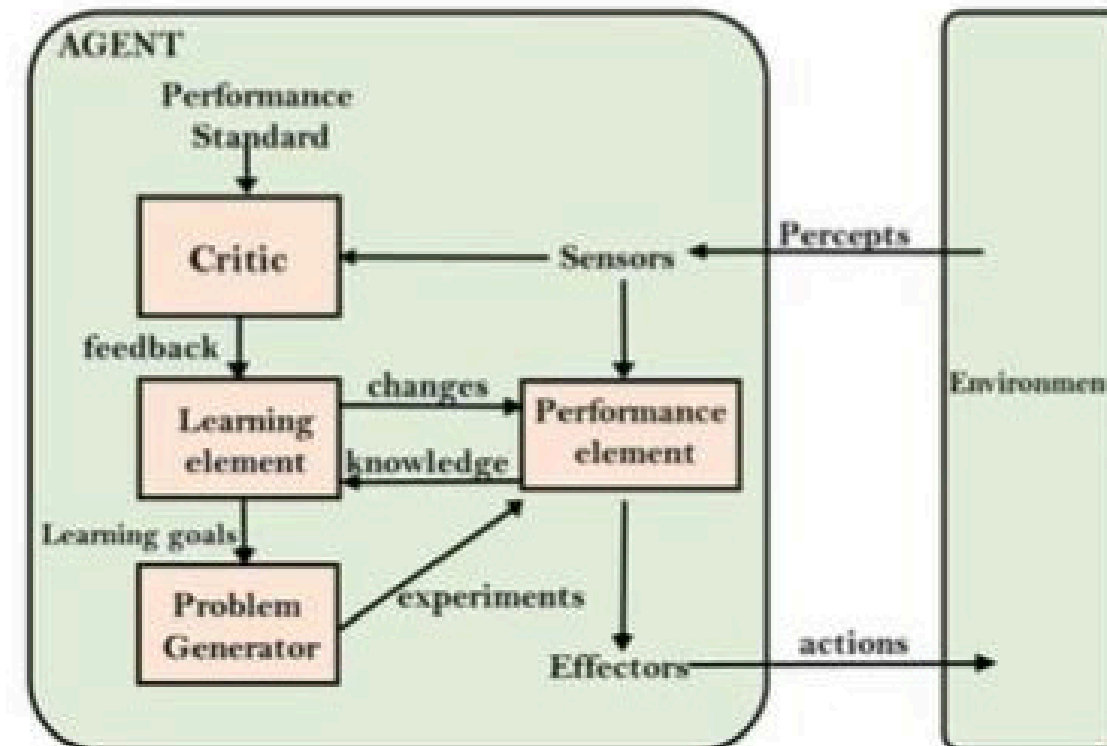
Forms of Learning

- **Learning Agent** can be thought of as containing a **Performance Element**, that decides, what actions to take, and a **learning element** that **modifies the performance element** so that it **makes better decisions**.



Learning Agent

- The design of a learning element is affected by three major issues:
 - Which *components* of the performance element are to be learned.
 - What *feedback* is available to learn these components.
 - What *representation* is used for the components.



Components of Learning Agents

- The components of learning agents include the following
 1. A direct mapping from conditions on the current state to actions.
 2. A means to infer relevant properties of the world from the percept sequence.
 3. Information about the way the world evolves and about the results of possible actions the agent can take.
 4. Utility information indicating the desirability of world states.
 5. Action-value information indicating the desirability of actions.
 6. Goals that describe classes of states whose achievement maximizes the agent's utility.

Automatic Taxi Driver



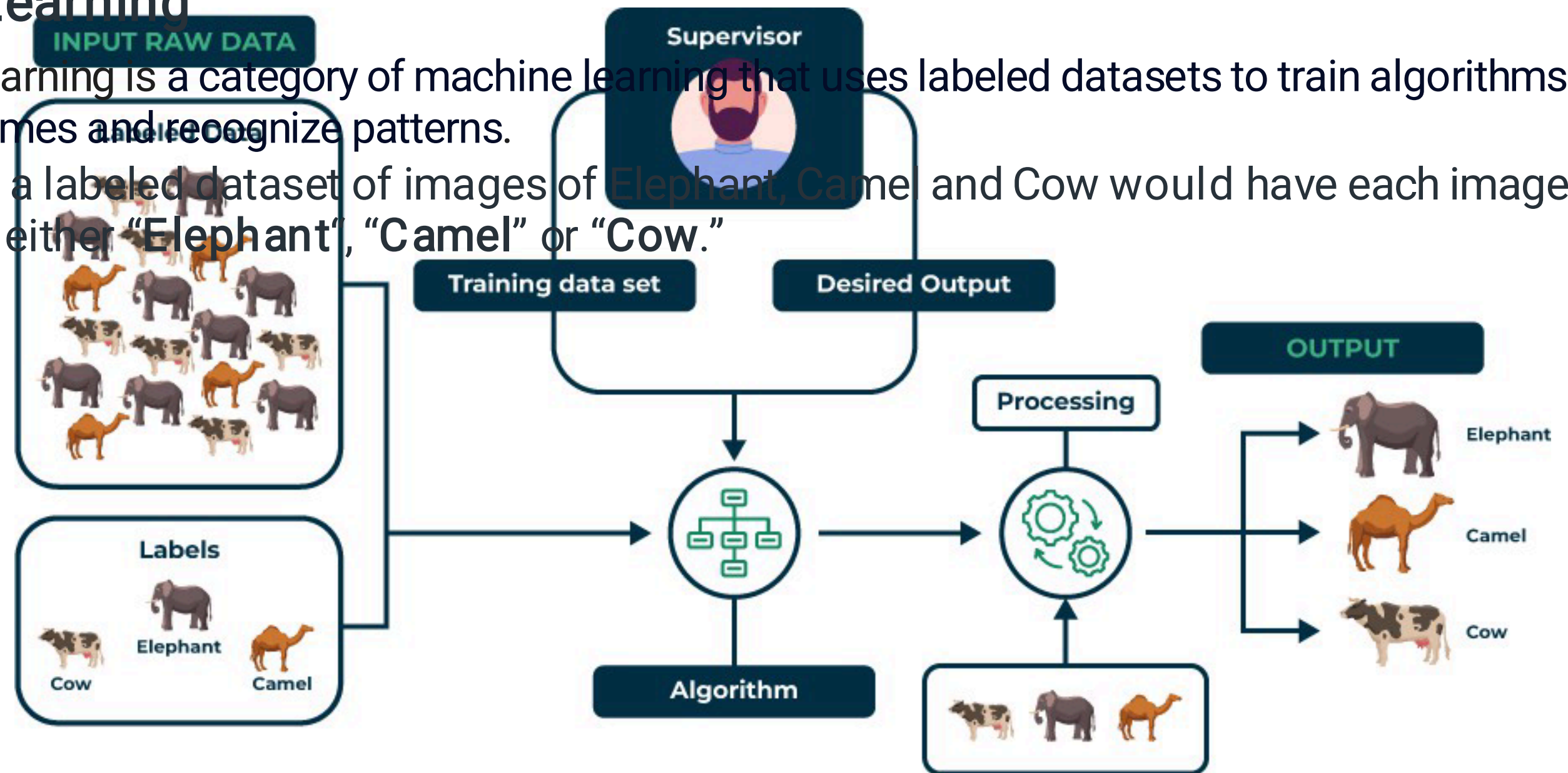
- Learning Agents' components can be learned from appropriate feedback.
- An agent training to become a **taxi driver**.
- Every time the instructor gives a command "Brake!", the agent can learn a **condition-action rule**, when we apply brake (component 1).
- By seeing many **camera images**, it can learn to recognize the objects on road (2).
- By trying **actions and observing the results**, for example, braking hard on a wet road, it can learn the effects of its actions (3).
- If there is **no tip from passengers**, but they shaken up during the trip, then it can learn a useful component of its **overall utility function** (4).
- The **type of feedback** available for learning is usually the most important factor in determining the nature of the learning problem that the agent faces.

Popular Machine Learning Algorithms

- The field of machine learning usually distinguishes three cases:
 - Supervised Learning
 - Unsupervised Learning
 - Reinforcement learning

Supervised Learning

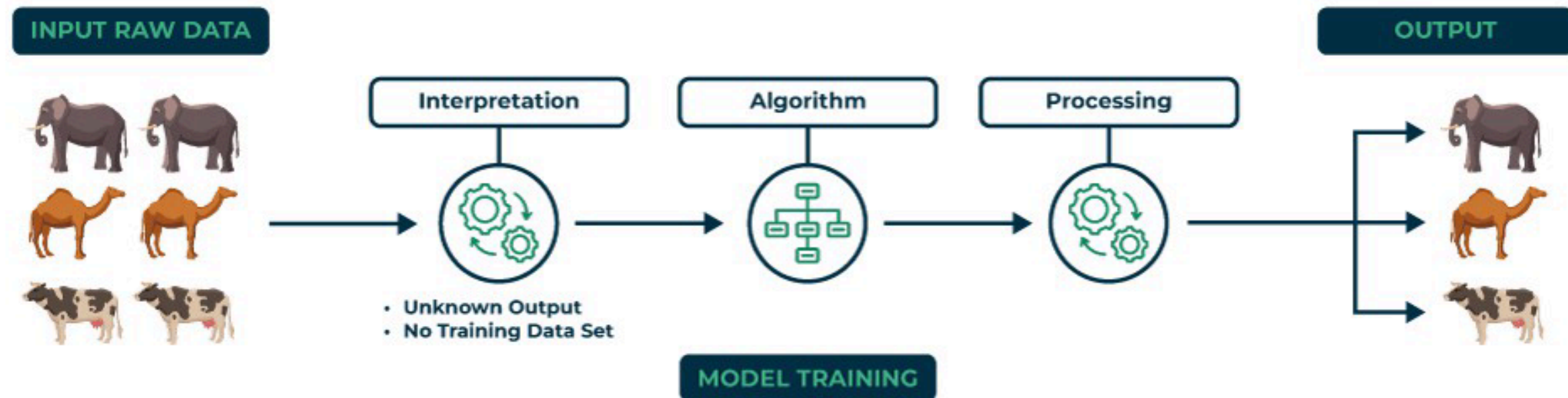
- Supervised learning is a category of machine learning that uses labeled datasets to train algorithms to predict outcomes and recognize patterns.
- For example, a labeled dataset of images of Elephant, Camel and Cow would have each image tagged with either “Elephant”, “Camel” or “Cow.”



- Types of Supervised Learning:** Supervised learning is classified into two categories of algorithms:
 - Regression:** A regression problem is when the output variable is a real value, such as “dollars” or “weight”.
 - Classification:** A classification problem is when the output variable is a category, such as “Yes” or “No”, “disease” or “no disease”.

Unsupervised Learning

- Unsupervised learning in artificial intelligence is a type of machine learning that learns from data without human supervision.
- Unsupervised learning is a type of machine learning that works with data that has no labels or categories. The main goal is to find patterns and relationships in the data without any guidance.
- For example, a unlabeled dataset of images of Elephant, Camel and Cow would have each image tagged with either “Elephant”, “Camel” or “Cow.”

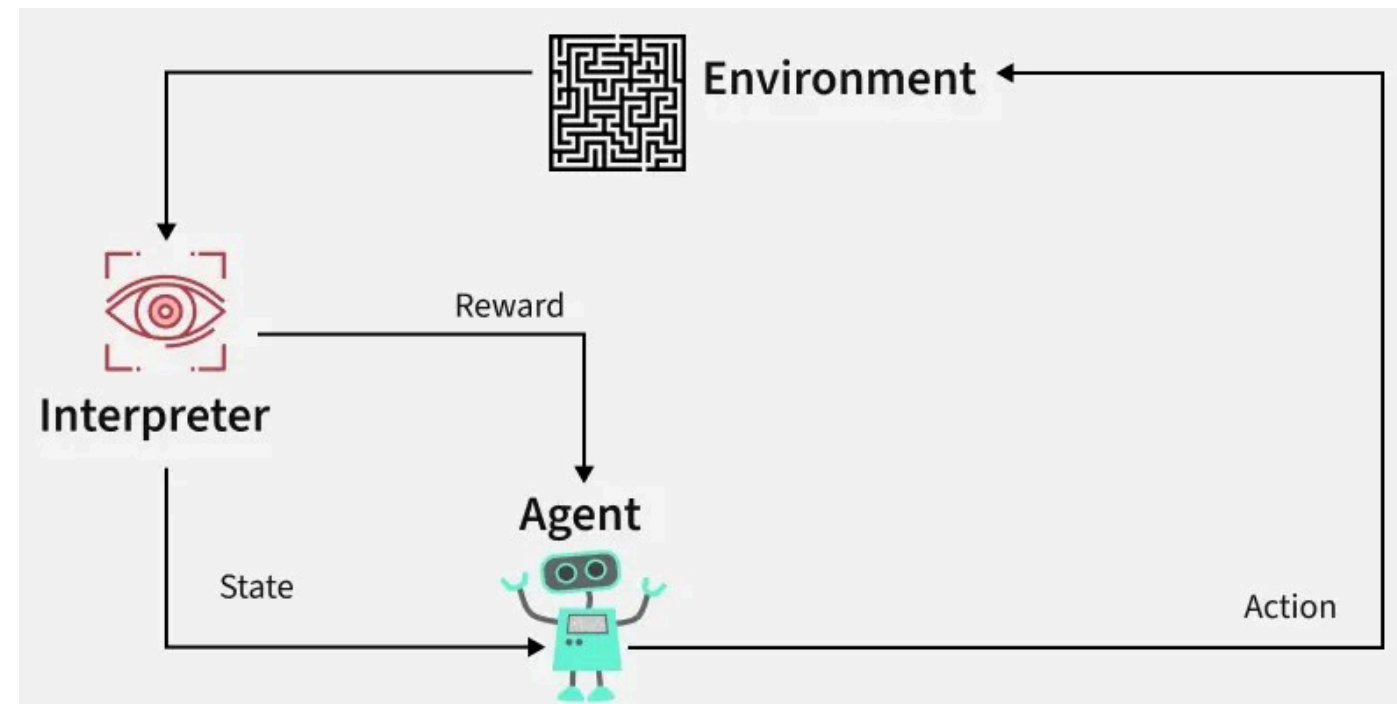


- **Types of Unsupervised Learning:** Unsupervised learning is classified into two categories of algorithms:
 - Clustering: A clustering problem is where you want to discover the inherent groupings in the data, such as grouping customers by purchasing behavior.
 - Association: An association rule learning problem is where you want to discover rules that describe large portions of your data, such as people that buy X also tend to buy Y

Reinforcement

Learning

Reinforcement Learning (RL) is a branch of machine learning that focuses on how agents can learn to make decisions through trial and error to maximize cumulative rewards. RL allows machines to learn by interacting with an environment and receiving feedback based on their actions. This feedback comes in the form of rewards or penalties.



- Reinforcement Learning revolves around the idea that an agent (the learner or decision-maker) interacts with an environment to achieve a goal. The agent performs actions and receives feedback to optimize its decision-making over time.
 - **Agent:** The decision-maker that performs actions.
 - **Environment:** The world or system in which the agent operates.
 - **State:** The situation or condition the agent is currently in.
 - **Action:** The possible moves or decisions the agent can make.
 - **Reward:** The feedback or result from the environment based on the agent's action.

Explanation Based Learning - EBL

- Definition
- Explanation-Based Learning (EBL) is a method for using available domain knowledge to improve supervised learning.
- EBL - can be in speed of learning, confidence of learning, accuracy of the learned concept, or a combination of these.
- EBL Computers uses the domain theory to inherit the properties of examples (part of Domain)

Explanation Based Learning - EBL

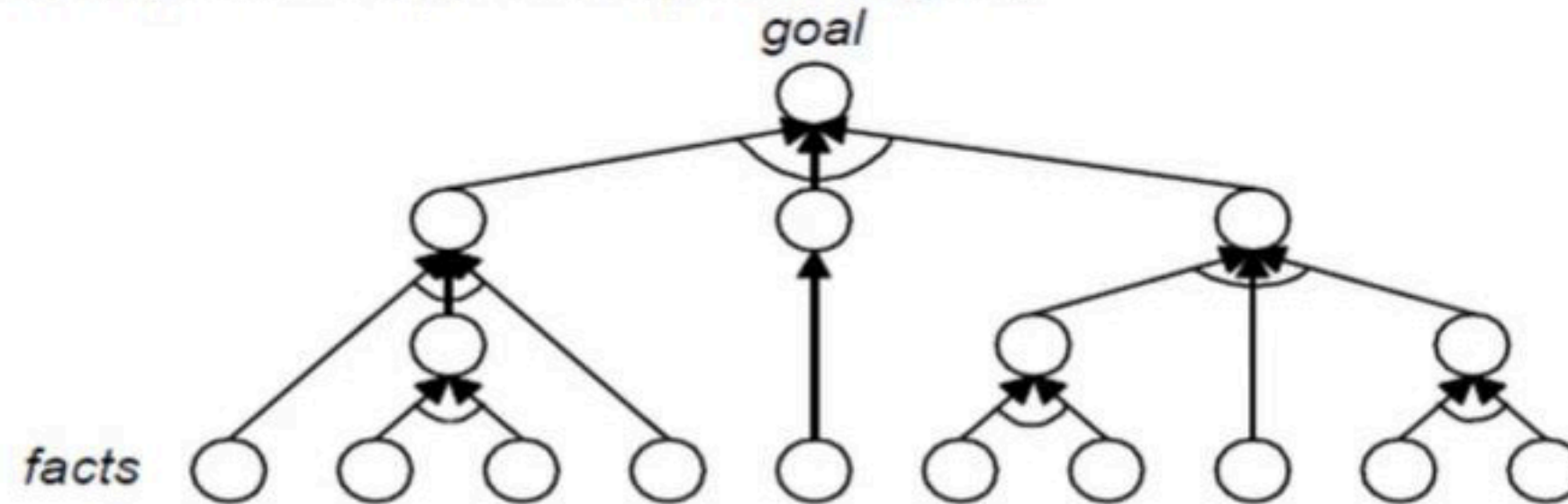
- EBL is a method for **extracting general rules** from Individual Observations.
- A part of domain is called an example, is a member of concept
- Learn the essential properties of the concept
- **Trade-off** - the need to collect many examples **for** the ability to “**explain**” single examples (a “domain” theory)

Explanation Based Learning - EBL ...

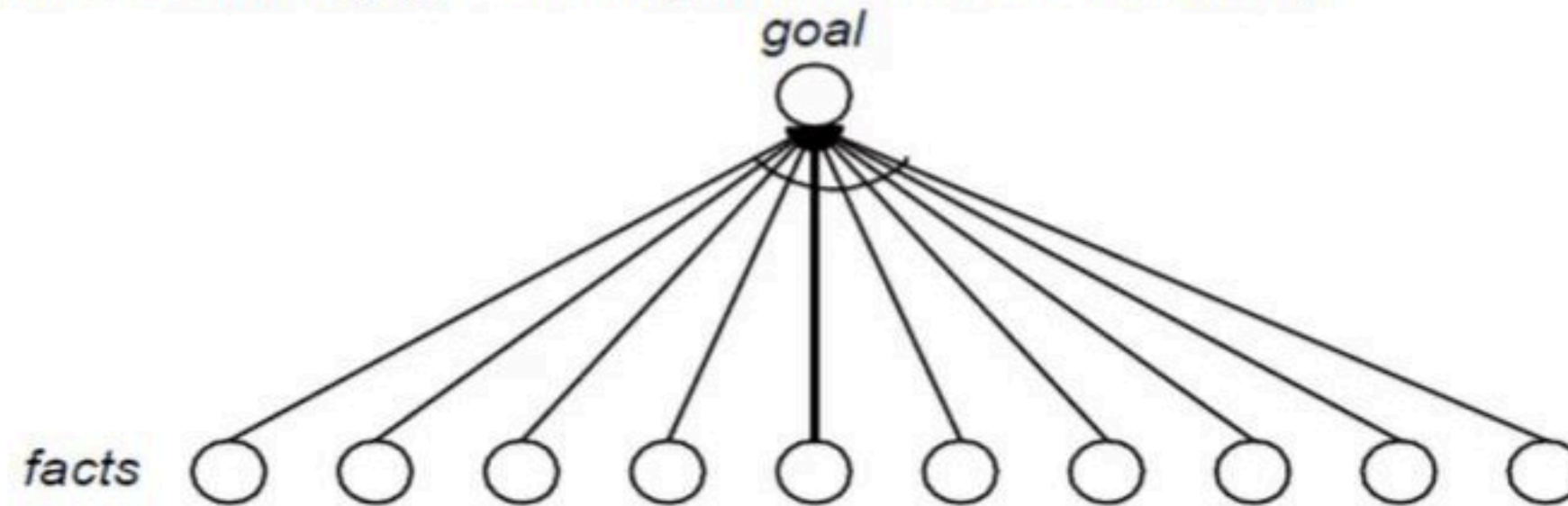
- An EBL accepts 4 kinds of input:
- A goal concept
 - a high level description of what the program is supposed to learn.
- A training example
 - what the learning sees in the world.
- A operational criterion (facts)
 - a description of which concepts are usable.
- A domain theory
 - a set of rules that describe relationships between objects and actions in a domain.
- Use problem solver to **justify**, using the *rules*, the *goal* in terms of the *facts*.
- **Generalization** - Generalize the justification as much as possible.

Standard Approach to EBL

An **Explanation** (detailed proof of goal)



After Learning (go directly from facts to solution):



Explanation Based Learning - EBL ...

- EBL computes a **generalization** of the training example, to describe the goal concept.
- It also satisfies the operational criterion.
- EBL has two steps:
- **Explanation**
 - -- the domain theory is used to remove all **unimportant aspects** of the training example with respect to the goal concept.
- **Generalization**
 - -- the explanation is **generalized as far possible** while describing the goal concept.◦

Statistical Learning

Statistics is the science of collecting, organizing, analyzing, interpreting, and presenting data. It encompasses a wide range of techniques for summarizing data, making inferences, and drawing conclusions.

- Statistical methods help quantify uncertainty and variability in data, allowing researchers and analysts to make data-driven decisions with confidence.

Types of Statistics

- There are commonly two types of statistics, which are discussed below:
 - Descriptive Statistics: It helps us simplify and organize big chunks of data. This makes large amounts of data easier to understand.

Statistical Learning

methods

- Applications of Statistics in Machine Learning

- Statistics is a key component of machine learning, with broad applicability in various fields.
- Feature engineering relies heavily on statistics to convert geometric features into meaningful predictors for machine learning algorithms.
- In image processing tasks like object recognition and segmentation, statistics accurately reflect the shape and structure of objects in images.
- Anomaly detection and quality control benefit from statistics by identifying deviations from norms, aiding in the detection of defects in manufacturing processes.
- Environmental observation and geospatial mapping leverage statistical analysis to monitor land cover patterns and ecological trends effectively.